

Frontend RH — l’application ressources humaines

React 19 · TypeScript 5.9 · Vite 7 · Tailwind CSS 4 · port **3001** Dossier : frontend-rh/

Prérequis : le socle technique commun (monorepo, axios, Providers, thème, build) est décrit dans 02-FRONTEND-PROJECTS.md , sections 1 à 5. Ce document se concentre sur ce qui est **spécifique à l’application RH**.

SOMMAIRE

- 1. [Le périmètre de l’application](#)
- 2. [La table de routage](#)
- 3. [La navigation en 5 familles](#)
- 4. [Module « Mon espace » — le point de vue employé](#)
- 5. [Module Gestion RH — employés et organigramme](#)
- 6. [Module Congés — le parcours complet](#)
- 7. [Module Administration — comptes, rôles, référentiels](#)
- 8. [Module Monitoring — l’AgentDesktop côté web](#)
- 9. [Le Dashboard RH](#)
- 10. [L’assistant IA RH](#)
- 11. [Particularités techniques de cette application](#)

1. Le périmètre de l’application

frontend-rh répond à **quatre familles de besoins** :

Famille	Utilisateur cible	Pages principales
Mon espace	Tout employé	Mon profil, mon calendrier, mes demandes
Gestion RH	Service RH, managers	Employés, organigramme, validations, dashboard
Administration	Administrateur	Comptes, rôles, référentiels, calendrier entreprise
Monitoring	Direction, RH	Suivi temps réel, rapports d’inactivité, historique agent

Différence fondamentale avec frontend-projects : cette application est centrée sur **la personne** (l’employé, son contrat, son temps, ses droits), alors que Projects est centrée sur **la production** (les projets, les livrables, les clients).

2. La table de routage

Extrait de frontend-rh/src/App.tsx :

PUBLIC		
/login	LoginPage	
/forgot-password	ForgotPasswordPage	
/reset-password	ResetPasswordPage	
PROTÉGÉ SANS PERMISSION (tout employé connecté)		
/change-password	ChangePasswordPage	← forcée si mustChangePassword
/dashboard	DashboardPage	
/mon-profil	MonProfilPage	
/mon-calendrier	MonCalendrierPage	
/mes-demandes	MesDemandesPage	

AVEC PERMISSION		
/employes	EmployesPage	VIEW_EMPLOYES
/employes/archives	ArchivesPage	VIEW_EMPLOYES
/organigramme	OrganigrammePage	VIEW_EMPLOYES
/admin/departements	DepartementsPage	VIEW_EMPLOYES
/demandes	DemandesPage	VIEW_DEMANDES
/demandes/new	NewDemandePage	VIEW_DEMANDES
/demandes/edit/:id	NewDemandePage	(libre – édition de sa demande)
/demandes/liste-papier	DemandesPapierPage	VIEW_VALIDATIONS
/mes-demandes/papier	MesDemandesPapierPage	VIEW_MES_DEMANDES
/validations	ValidationsPage	VIEW_VALIDATIONS
/referentiels	ReferentielsPage	VIEW_REFERENTIELS
/calendrier	CalendrierPage	VIEW_CALENDRIER
/restrictions-conges	RestrictionsCongesPage	VIEW_RESTRICTION_CONGE
/comptes	ComptesPage	VIEW_COMPTES
/roles	RolesPage	VIEW_ROLES
/dashboard-rh	DashboardRHPage	VIEW_DASHBOARD
/suivi-temps-reel	SuiviTempsReelPage	VIEW_MONITORING
/rapports-inactivite	RapportsInactivitePage	VIEW_MONITORING
/historique-agent	HistoriqueAgentPage	VIEW_MONITORING

La redirection croisée vers la page de connexion

```
const ProtectedRoute = ({ children }) => {
  const { isAuthenticated } = useAuth();
  if (!isAuthenticated) {
    const projectsAppUrl = import.meta.env.VITE_PROJECTS_APP_URL?.trim();
    if (projectsAppUrl) {
      window.location.replace(`${projectsAppUrl}/login`);
      return <div>Redirection vers la page de connexion...</div>;
    }
    return <Navigate to="/login" replace />;
  }
  return <>{children}</>;
};
```

Décision de conception à expliquer : il existe **une seule vraie page de connexion** dans tout le système, celle de `frontend-projects` . Si un utilisateur arrive sur l'application RH sans être connecté, il est renvoyé vers elle. Cela évite de maintenir trois pages de connexion différentes et garantit un point d'entrée unique.

3. La navigation en 5 familles

`Sidebar.tsx` de RH organise le menu en **cinq groupes** :

1. ACCUEIL	→ /dashboard	
2. MON ESPACE	→ /mon-calendrier	
└─ Mon calendrier		VIEW_MON_CALENDRIER
└─ Mes demandes		VIEW_MES_DEMANDES
└─ Demandes congés	→ /mes-demandes	
└─ Demandes papiers	→ /mes-demandes-papier	
3. GESTION RH	→ /employes	
└─ Employés		VIEW_EMPLOYES
└─ Liste des employés	→ /employes	
└─ Organigramme	→ /organigramme	
└─ Archives	→ /employes/archives	
└─ Validation demandes		VIEW_VALIDATIONS
└─ Demandes congés	→ /demandes	
└─ Demandes papiers	→ /demandes/liste-papier	
└─ Dashboard RH		VIEW_DASHBOARD
└─ Départements		VIEW_EMPLOYES
4. ADMINISTRATION	→ /comptes	
└─ Comptes		VIEW_COMPTES
└─ Rôles		VIEW_ROLES
└─ Référentiels		VIEW_REFERENTIELS
└─ Calendrier entreprise		VIEW_CALENDRIER
└─ Restrictions de congés		VIEW_RESTRICTION_CONGE
5. MONITORING	→ /suivi-temps-reel	VIEW_MONITORING
└─ Suivi temps réel		
└─ Rapports		
└─ Historique agent		

Chaque famille et chaque sous-entrée porte sa permission. Un employé sans droit particulier ne voit que **Accueil** et **Mon espace**.

4. Module « Mon espace »

C’est le point de vue de l’employé lambda — celui qui n’a aucune permission d’administration. Trois pages :

MonProfilPage.tsx

Ses propres informations : identité, poste, département, contrat, manager, solde de congé, compétences, documents.
Modification du mot de passe.

MonCalendrierPage.tsx

Son calendrier personnel : ses congés approuvés, ses jours de télétravail, les jours fériés de l’entreprise, les périodes bloquées.

MesDemandesPage.tsx + MesDemandesPapierPage.tsx

Historique de ses demandes avec leur statut, et le suivi de ses demandes administratives (attestations...).

ChangePasswordPage.tsx — la page forcée

Quand `Compte.mustChangePassword === true` (cas d’un compte fraîchement créé dont les identifiants ont été envoyés par email), l’application redirige vers cette page et **n’en laisse pas sortir** avant le changement.

5. Module Gestion RH

EmployesPage.tsx — 1 824 lignes, la page la plus riche de l’application

Elle couvre le cycle de vie complet d’un employé :

Bloc	Champs / actions
Identité	Nom, prénom, email, téléphone, téléphone pro, genre, photo
Documents légaux	CIN (8 chiffres , validé), CNSS, RIB bancaire (20 chiffres , validé)
Contrat	Poste, département, type de contrat, date d’embauche, date de fin
Rémunération	Salaire, <code>modeSalaire</code> (BRUT ou NET)
Hiérarchie	Manager (auto-relation) → construit l’organigramme
Congés	Solde courant, solde initial (reprise de l’existant)
Compétences	Liste éditable
Documents	Upload de pièces jointes (contrat, diplômes...)
Drive	Lien vers son dossier
Archivage	Bouton « Archiver » plutôt que supprimer

Le champ `modeSalaire` est directement lié à la paie : si l’employé est en mode `NET`, le backend lance une **recherche dichotomique** pour retrouver le brut correspondant (voir `01-BACKEND.md` §12.3).

OrganigrammePage.tsx

Rendu visuel de la hiérarchie, construit à partir de la relation `Employe.manager` ↔ `Employe.subordonnes`.

ArchivesPage.tsx

Les employés archivés (`archived = true`). **Ils ne sont jamais supprimés** : leurs bulletins de paie, congés et historiques doivent rester consultables pour des raisons légales.

DepartementsPage.tsx

Gestion des départements (alimente le référentiel `DEPARTEMENT`).

6. Module Congés

C'est le parcours métier le plus complet de l'application RH.

6.1 NewDemandePage.tsx (987 lignes) — la création de demande

Le formulaire s'adapte au **type de demande** choisi :

TypeDemande = CONGE | AUTORISATION | TELETRAVAIL

Type	Champs affichés
CONGE	Date début, date fin, type de congé, raison, justificatif
AUTORISATION	Date, heure début, heure fin, raison (quota mensuel affiché)
TELETRAVAIL	Date début, date fin, raison

Le calcul en temps réel des jours effectifs :

L'utilisateur saisit 10/03 → 14/03

↓

POST /api/demandes/compute-effective-days

↓

Le backend :

- retire les samedis et dimanches (selon HoraireTravail.joursTravail)
- retire les jours fériés (table `calendrier`)
- applique les règles particulières (congé maladie, vendredis seuls...)

↓

Réponse : { joursEffectifs: 5, detail: [...] }

↓

Affichage immédiat : « 5 jours ouvrés seront décomptés de votre solde »

Pourquoi ce calcul est-il fait côté backend et pas dans le navigateur ? Parce qu'il dépend de données serveur (jours fériés de l'entreprise, horaire affecté à cet employé, types de congé paramétrés) **et** parce que le résultat doit être identique à celui utilisé au moment de l'approbation. Le calculer deux fois avec deux implémentations différentes créerait des écarts.

Avant validation, la page vérifie aussi : - Le **solde disponible** (GET /api/demandes/solde-conge/{employeId}) - Les **périodes bloquées** (campagnes clients, inventaires) - Le **quota d'autorisations** du mois (120 minutes par défaut)

6.2 ValidationsPage.tsx / DemandesPage.tsx — le côté validateur

Le manager ou le RH voit les demandes en attente et peut : - **Approuver** (unitairement ou en lot — batchApprove) - **Refuser** avec un motif obligatoire - Consulter l'**historique complet** des changements de statut

Chaque décision déclenche une notification vers l'employé.

6.3 DemandesPapierPage.tsx / MesDemandesPapierPage.tsx

Circuit distinct pour les demandes administratives (TypeDemande.ADMINISTRATION) : attestation de travail, attestation de salaire, etc. Elles ne consomment pas de solde et suivent un flux « demandé → préparé → à retirer ».

6.4 RestrictionsCongesPage.tsx

Définition des PeriodeBloquee : périodes pendant lesquelles aucun congé ne peut être posé (ex. lancement d'une campagne majeure).

6.5 CalendrierPage.tsx — le calendrier de l'entreprise

Saisie des jours fériés (TypeJour , OrigineJour). Ces jours sont ensuite consommés par : - le calcul des jours effectifs de congé, - le calcul des jours ouvrés de la paie, - l'AgentDesktop (pas d'absence comptée un jour férié).

7. Module Administration

Création d'un compte pour un employé. À la création : 1. Un mot de passe temporaire est généré, 2. `EmailService.sendCredentials()` l'envoie par mail, 3. `mustChangePassword = true` force le changement à la première connexion.

Actions : activer / désactiver, réinitialiser le mot de passe, affecter des rôles.

`RolesPage.tsx` — le cœur du système de droits

L'administrateur : - crée un rôle (ex. « Social Media Junior »), - coche les permissions parmi le catalogue (26+), - affecte le rôle à des comptes.

Rien n'est codé en dur. Un nouveau profil métier ne demande **aucun redéploiement** : c'est de la configuration en base.

Question probable du jury : « Comment gérez-vous un nouveau type d'utilisateur ? » **Réponse** : « Je crée un rôle dans l'interface et je coche les permissions nécessaires. Aucune ligne de code. Le rôle ADMIN, lui, reçoit automatiquement toutes les permissions existantes au démarrage — y compris celles ajoutées après. »

`ReferentielsPage.tsx`

Édition de toutes les listes paramétrables : genres, types de contrat, départements, postes, niveaux hiérarchiques, sites, types de demande, types de congé, formats et plateformes de media plan, catégories de revenu et de charge — **et les paramètres système** (`MAX_AUTORISATION_MINUTES` , `SOLDE_CONGE_AN1` , `SOLDE_CONGE_AN2_PLUS`).

8. Module Monitoring

C'est la face web de l'**AgentDesktop** (application Electron installée sur les postes — voir `01-BACKEND.md §13`).

`SuiviTempsReelPage.tsx`

Tableau de bord en direct : qui est connecté, qui est actif, qui est inactif, depuis quelle IP / quel réseau Wi-Fi. Alimenté par les `heartbeats` envoyés toutes les 60 secondes par l'agent.

`RapportsInactivitePage.tsx`

Liste des périodes d'inactivité détectées (> 2 minutes sans souris ni clavier), soumises à **décision managériale** (`RapportDecisionRequest`) : le manager confirme ou justifie.

Point important à défendre en soutenance : « Le système détecte, il ne sanctionne pas. Chaque rapport d'inactivité passe par une validation humaine. C'est un choix éthique et aussi une nécessité pratique : une inactivité peut être une réunion, un appel téléphonique ou un déplacement. »

`HistoriqueAgentPage.tsx`

Historique des pointages : heures d'entrée et de sortie, y compris celles **fermées automatiquement** par `PointageScheduledService` quand un poste s'éteint sans déconnexion propre (l'heure retenue est celle du dernier heartbeat, pas l'heure courante).

9. Le Dashboard RH

`DashboardPage.tsx` — accueil de tout employé

Vue synthétique : ses tâches obligatoires, ses demandes en cours, ses notifications, le calendrier du mois.

`DashboardRHPage.tsx` (768 lignes) — vue direction

Indicateurs agrégés avec **Recharts** : - Effectif par département / par type de contrat - Répartition hommes / femmes - Congés en cours et à venir - Demandes en attente de validation - Contrats arrivant à échéance (alimenté par `ContratExpirationScheduler`) - Taux de présence

10. L’assistant IA RH

Un assistant global, contrairement à Projects

Dans l’application RH, le widget est monté dans le **layout** (donc présent sur toutes les pages), et le déclencheur est une entrée de la barre latérale « Assistant RH ».

```
<AiChatWidget
  apiUrl={API_BASE}
  getAuthToken={getAccessToken}
  onUnauthorized={handleUnauthorized}
  capability="GENERAL"
  isAdmin={isAdmin}
  launcherLabel="Assistant RH"
  hideLauncher // le déclencheur vit dans la Sidebar
  theme={{ mode: theme, light: {...}, dark: {...} }}
/>
```

Ce que le commentaire du code explique

« Assistant unique : il remplace à la fois l’ancien “Assistant RH” (chatbot Groq/Llama distinct, basé sur le règlement intérieur) et le widget “Assistant paie”. La capacité GENERAL couvre déjà les questions RH générales (congs, règlement intérieur) et peut mobiliser les outils de paie à la demande : le classement d’intention du backend route chaque message indépendamment de cette valeur. »

Ce qu’il faut retenir : `capability="GENERAL"` ne verrouille rien. Le backend **classe l’intention de chaque message** et choisit lui-même les outils. Cette propriété ne sert qu’à orienter les **suggestions affichées à l’ouverture** du chat.

Évolution du projet à raconter : il y avait d’abord deux chatbots séparés (un RH sur Groq/Llama pour le règlement intérieur, un pour la paie). Ils ont été **fusionnés en un assistant unique** avec routage automatique d’intention. C’est une vraie décision d’architecture, pas un accident.

`AiAssistantContext.tsx` — le pilotage depuis n’importe quelle page

Un contexte React expose :

```
{ open, setOpen, pendingMessage, consumePendingMessage }
```

Ce qui permet à **n’importe quelle page** d’ouvrir le chat avec une question pré-remplie. Exemple : un bouton « Expliquer mon bulletin » sur la fiche de paie ouvre l’assistant avec la question déjà écrite.

Le trou de sécurité comblé par le widget

```
/**
 * Jeton absent ou expiré : on purge et on renvoie vers la connexion, exactement
 * comme l'intercepteur axios. Le backend répondant 403 (et non 401) dans ce cas,
 * cet intercepteur ne se déclenche pas : le widget comble ce trou pour son propre
 * périmètre.
 */
```

Détail à citer : les endpoints IA renvoient **403** (et non 401) quand le jeton est absent. L’intercepteur axios, qui n’écoute que le 401, ne se déclencherait pas. Le widget gère donc son propre cas, avec exactement le même comportement.

11. Particularités techniques

11.1 Les dépendances propres à RH

Dépendance	Usage
antd 5.24	Tableaux et formulaires complexes (page Employés)
three + @react-three/fiber + @react-three/drei	Logo 3D animé
moment	Manipulation de dates dans les calendriers
recharts	Graphiques du Dashboard RH

11.2 AnimatedThemeToggler.tsx

Bascule clair/sombre animée. Le thème est stocké dans un **cookie partagé** (et non `localStorage`), de sorte qu'il suive l'utilisateur d'une application à l'autre.

11.3 hooks/useConfirm.ts

Transforme un dialogue de confirmation en promesse :

```
const confirm = useConfirm();
if (await confirm('Archiver cet employé ?')) { /* ... */ }
```

Beaucoup plus lisible que la gestion manuelle d'un état `isModalOpen`.

11.4 hooks/useTachesObligatoires.ts

Charge les tâches obligatoires de l'employé et les affiche en permanence, quelle que soit la page.

11.5 components/ui/DataTable.tsx

Tableau générique et typé, réutilisé sur la quasi-totalité des pages de liste : tri, filtrage, pagination, actions par ligne.

11.6 components/ui/TimePicker.tsx

Sélecteur d'heure sur mesure, utilisé pour les **autorisations de sortie** (heure de début / heure de fin) et les horaires de travail.

11.7 Configuration

```
VITE_APP_KIND=rh
VITE_API_URL=http://localhost:8080
port dev : 3001
base de build : /Antigone_RH/rh/
```